PXP Client Portal — Technical Documentation

> **Living document.** This is the canonical, always-current explanation of how the portal works (backend + frontend). Update it whenever the system changes — see Maintaining this document and the Changelog at the bottom.

Last updated: 2026-06-22

---

1. What this is

The PXP Client Portal is a web app that lets PXP Solutions' customers log in and track their orders in real time — production status, ship dates, live carrier tracking — and message their account executive (AE). PXP staff use the same app to manage clients and answer messages.

It reads order data **directly from the ShopWorks MySQL database** (read-only) and stores its own small amount of data (logins, access rules, messages) separately.

---

2. Architecture at a glance

- **One server**, DigitalOcean droplet at `143.198.14.93`, app at `/opt/pxp-order-dashboard`.
- The **FastAPI backend** both serves the API and hosts the compiled React frontend (so there's a single origin, no CORS issues in production).
- Managed by **systemd** as `pxp-portal.service`.

---

3. Tech stack

Layer	Technology
Frontend	React 18, Vite 5, Tailwind CSS 3, React Router
Backend	Python, FastAPI, Uvicorn
Auth	JWT (PyJWT, HS256, 8-hour tokens), bcrypt password hashing
Databases	MySQL (`shopworks` read-only + `local_reference`), SQLite (messages)
External APIs	UPS Tracking (live tracking), Microsoft Graph (transactional email)
Hosting	DigitalOcean droplet, systemd service, FastAPI static file serving
Fonts/Theme	DM Sans + Fira Code, brand palette matched to pxpsolutions.com

---

4. Backend

Located in `backend/`. Each file has one clear job.

File	Responsibility
`main.py`	All API routes, app setup, startup tasks, serves the SPA
`auth.py`	JWT create/verify, bcrypt hashing, role dependencies
`db.py`	MySQL connection pool (5 connections, user `swro`)
`cache.py`	In-memory TTL cache for order lists
`ups.py`	UPS Tracking API client (OAuth client-credentials)
`messages.py`	SQLite-backed client↔AE messaging store
`email_utils.py`	Invite / reset / error-alert emails via Microsoft Graph
`migrate_invite.py`	One-off migration helper (historical)

4.1 Startup tasks (`main.py`)

On boot the app runs three `@app.on_event("startup")` hooks: 1. **`ensure_invite_schema`** — adds any missing columns/tables to `local_reference` (invite tokens, `is_super_admin`, `view_all_orders`, AE/company access tables). 2. **`init_messages_db`** — creates the SQLite messages table if absent. 3.

`warm_order_cache` — a background thread (starts ~4s after boot) pre-loads the all-companies order list and each client company's list into cache, so the first login after a deploy/restart is fast instead of hitting a cold query.

4.2 Authentication & roles

- Login (`POST /api/auth/login`) checks the bcrypt hash and returns a **JWT** holding `sub` (email), `client_id`, `company_name`, `is_admin`, `is_super_admin`, `view_all_orders`, and `exp` (8h).
- Four access tiers:

Tier	Sees	Notes
Regular client	Their company's orders (optionally narrowed to specific contact names)	Default user
Company admin	Same as client + can manage their own company's users	`is_admin`
Internal PXP Staff	Orders across companies, filtered by assigned AE names and/or companies	`view_all_orders` (sales/AEs)
Super admin	Everything, all companies, all clients	`is_super_admin`

- Access filtering is centralized in `_build_filter_clause`. AE access and company access combine as **OR** (a staff user sees an order if its AE matches *or* its company matches one of their grants).

4.3 Orders

- `GET /api/orders` returns one row **per order** (designs for a multi-design order are combined into that single row). Supports filters (PO, contact, design, type, ship-date range, shipped status) and a `year` parameter (defaults to current year; index-friendly half-open date range).
- Status fields from ShopWorks are **codes, not booleans**: `1` = done, `0` = pending, `0.5` = partial, `222` = not required, `8` = N/A. The API exposes both booleans (e.g. `shipped`) and raw codes (`shipped_code`, etc.), plus a derived `closed` flag (shipping resolved: shipped / N/A / not required).
- Results are cached in memory per `company:year` (90-min TTL), skipped when filters or AE/company restrictions are active. Cache is busted on any admin data change.

4.4 Live tracking (`ups.py`)

- `GET /api/orders/{order_number}/tracking` — **access-checked** (same filter as the orders list, so a client can only pull tracking for their own orders). Returns normalized status, delivery date, and recent scan events for the order's UPS (`1Z`) numbers.
- UPS uses OAuth client-credentials; the token is cached in memory and auto-refreshed.
- Tracking results are cached in memory (1h TTL; delivered shipments cached as final) to bound API calls.
- FedEx/USPS currently fall back to "track on carrier site" links. **FedEx live tracking is planned** (sandbox validated; awaiting production keys).

4.5 Messaging (`messages.py`)

- Clients and company admins send messages to their AE; PXP staff (super admins + Internal PXP Staff) read and reply from an inbox.
- Stored in **SQLite** (`backend/data/messages.db`) — the app owns this file, so it needs no MySQL privileges. A conversation is all rows sharing a `client_id`.
- Inbox and replies are **access-scoped by company** (super admins see all; staff see their assigned companies). Unread counts power the badges.

4.6 Admin

Endpoints under `/api/admin/*` manage clients (create/invite, update, delete), contacts, AE access, company access, and (super-admin only) system logs + cache. Company admins are restricted to their own company; super admins are unrestricted.

4.7 Full endpoint list

Method	Path	Purpose
POST	`/api/auth/login`	Log in, returns JWT
POST	`/api/auth/set-password`	Activate account / reset via invite token
GET	`/api/auth/invite/{token}`	Validate an invite token
POST	`/api/auth/forgot-password`	Request a reset email
GET	`/api/orders`	Order list (filters + year)
GET	`/api/filters`	Distinct order types for filter dropdown
GET	`/api/orders/{order_number}/tracking`	Live UPS tracking for an order
POST	`/api/messages`	Client sends a message
GET	`/api/messages/mine`	Client's own thread
GET	`/api/messages/unread`	Unread count (client or staff)
GET	`/api/messages/inbox`	Staff inbox (conversation summaries)
GET	`/api/messages/thread/{client_id}`	Staff opens a conversation
POST	`/api/messages/thread/{client_id}/reply`	Staff replies
GET	`/api/admin/clients`	List clients

POST	`/api/admin/clients`	Create / invite a client
PUT	`/api/admin/clients/{id}`	Update a client
DELETE	`/api/admin/clients/{id}`	Delete a client
POST	`/api/admin/clients/{id}/resend-invite`	Resend invite
GET/POST	`/api/admin/clients/{id}/contacts`	List / add contact access
DELETE	`/api/admin/contacts/{mapping_id}`	Remove contact access
GET/POST	`/api/admin/clients/{id}/ae-access`	List / add AE access
DELETE	`/api/admin/ae-access/{mapping_id}`	Remove AE access
GET/POST	`/api/admin/clients/{id}/company-access`	List / add company access
DELETE	`/api/admin/company-access/{mapping_id}`	Remove company access
GET	`/api/admin/suggest-contacts`	Contact autocomplete
GET	`/api/admin/suggest-aes`	AE-name autocomplete
GET	`/api/admin/companies`	Company autocomplete
GET	`/api/admin/cache` / POST `/api/admin/cache/bust`	Cache info / clear (super admin)
GET	`/api/admin/logs`	App logs (super admin)
GET	`/api/health`	Health check
GET	`/{full_path}`	Serves the React SPA (index.html no-cache)

---

5. Frontend

Located in `frontend/`. React SPA built by Vite into `frontend/dist/`, which the backend serves.

5.1 Pages (`src/pages/`)

Page	Route	Who	Purpose
`Login.jsx`	`/login`	All	Sign in
`Dashboard.jsx`	`/dashboard`	All logged in	Orders table, filters, stats, tracking, chat

`AdminPage.jsx`	`/admin`	Admins	Manage clients, access, invites
`MessagesInbox.jsx`	`/messages`	Super admin + Internal Staff	Read/reply to client messages
`SetPassword.jsx`	`/set-password`	Invited users	Create/reset password
`ForgotPassword.jsx`	`/forgot-password`	All	Request reset link

5.2 Components (`src/components/`)

Component	Purpose
`OrderTable.jsx`	The 7-column orders table + mobile cards, sorting, pagination (20/page), status badges, the `parseDesigns`/`isOverdue` helpers
`OrderDetailDrawer.jsx`	Slide-in panel with full order detail, per-design status, and live UPS tracking
`StatsBar.jsx`	Compact stat pills (Total / In Progress / Shipped / On Hold / Overdue)
`Sidebar.jsx`	Filter panel (desktop sidebar + mobile drawer)
`ChatWidget.jsx`	Floating client chat: rule-based "Help" bot (no API/AI) + two-way "My AE" messaging

5.3 API client (`src/api.js`)

A thin `fetch` wrapper. Stores the JWT and role flags in `localStorage`, attaches the `Authorization: Bearer` header, and redirects to `/login` on a 401. One exported function per endpoint.

5.4 Table design

The table shows **7 curated columns** (Order #, Design, Order Type, Req Ship, Status, Tracking #, Customer) so it fits without horizontal scrolling. Everything else (all fields, per-design status, production timeline, live tracking) lives in the detail drawer opened by clicking a row. CSV export still includes all fields.

5.5 Theme

Matched to **pxpsolutions.com**: DM Sans typography, a sky-blue accent (`#29ABE2`), solid fills (no gradients), softened corners, navy (`#15233B`) headers, and tasteful motion (entrance stagger, hover lifts, live status dots, a floating chat bubble). `prefers-reduced-motion` is respected. Theme tokens live in `tailwind.config.js`; global animations and component classes in `src/index.css`.

---

6. Data & databases

- **`shopworks`** (read-only via the `swro` MySQL user): the live order data — `orders`, `orderdes` (designs), `pack_import` (tracking numbers), `employee` (AEs), `manifest` (carriers), `location`, plus `local_reference.order_type` for readable type names.
- **`local_reference`** (read + write on specific tables): portal-owned tables — `clients`, `client_contacts`, `client_ae_access`, `client_company_access`. Note: the `swro` user has write access only on these named tables (granted explicitly); it cannot `CREATE` tables, which is why messaging uses SQLite.
- **SQLite** (`backend/data/messages.db`): client↔AE messages.

---

7. Deployment & operations

- **Server:** DigitalOcean droplet `143.198.14.93`, app at `/opt/pxp-order-dashboard`.
- **Service:** `systemctl restart pxp-portal.service` (Uvicorn on port 8080).
- **Deploy flow:**
- Backend change → `scp backend/<file>.py root@143.198.14.93:/opt/pxp-order-dashboard/backend/` → restart service.
- Frontend change → `npm run build` → `scp -r frontend/dist/* root@…/frontend/dist/` (no restart needed; FastAPI serves the new files, and `index.html` is sent with `no-cache` so browsers always get the latest bundle).
- **Logs:** `backend/logs/app.log` (rotating), also visible to super admins in-app.
- **Config:** `backend/.env` (never committed — see `.env.example` for the keys: MySQL creds, `JWT_SECRET`, Microsoft Graph creds, `PORTAL_URL`, UPS creds).

---

8. Security notes

- Passwords hashed with bcrypt; secrets only in the server `.env` (git-ignored).
- Order data is **read-only** from ShopWorks; the portal never writes to `shopworks`.
- Every data endpoint is access-checked against the caller's JWT role and company.
- Rate limiting on auth endpoints (login, forgot-password).
- `prefers-reduced-motion` and contrast-checked button colors for accessibility.

---

9. Maintaining this document

When you change the system, update this file in the **same commit** as the change:

- New endpoint → add it to §4.7.
- New page/component → add it to §5.
- New external integration, table, or env var → update the relevant section.
- Always add a dated line to the Changelog and bump "Last updated".

The PDF (`docs/PXP-Client-Portal-Documentation.pdf`) is generated from this file — regenerate it after meaningful updates (see `docs/` for the generator note).

Changelog

- **2026-06-22** — Initial living documentation written. Covers current system: AE/company access, year selector, overdue tracking, one-row-per-order table with accurate ShopWorks status codes, per-design status, pagination, in-memory caching + startup warming, live UPS tracking, client↔AE messaging (SQLite), admin management, PXP branding, and the pxpsolutions.com-matched theme with tasteful motion. FedEx live tracking is planned (sandbox validated; awaiting production keys).